

Skyrmion Simulator: C++ Port

Function Equivalence, User Guide, and Profiling

Rui Barreira

October 1, 2026

1 Overview

The micromagnetic SAF skyrmion simulator under `src/skyrmion_simulator/simulator/` (Python/NumPy) has been ported to C++17 under `src/simulator_cpp/`. The C++ build produces a deterministic simulator (`skyrmion_sim`), one binary per sweep source, and a parity test-suite that checks every ported function against its Python counterpart to machine precision. Outputs remain NumPy `.npz` archives, so all Python post-processing, plotting, and animation scripts read the C++ results unchanged.

This document lists the Python $\leftrightarrow$ C++ function equivalence, explains how to build and configure runs, and reports a like-for-like performance comparison on a 64×64 lattice.

2 Function equivalence

Table 1 maps each Python function to its C++ counterpart. Where several small Python field builders are fused into one C++ loop (for cache efficiency) this is noted. The deterministic simulator, the sweep layer, and the full stochastic-LLG path (thermal noise, the Stratonovich–Heun stepper, connected-component diagnostics, production scans, and the physics-validation gates) are all ported. The analytic Thiele helpers and all plotting intentionally remain on the Python side.

Error contract. The port follows a strict “no silent errors” rule: every invalid input raises. Four cases where the Python original previously emitted a silent NaN (or accepted an out-of-range value) were aligned so Python and C++ now raise identically: `skyrmion_center` on an empty core mask, `uniform_state` on a zero-magnitude direction, `normalize` on a zero-magnitude spin, and `skyrmion_profile` on a polarity outside $\{+1, -1\}$.

Table 1: Python to C++ function equivalence.

Python (module.function)	C++ (symbol)	C++ file
<i>Parameters</i>		
<code>parameters.default_params</code>	<code>make_default_params</code>	<code>parameters.cpp</code>
<code>parameters._precompute</code>	<code>precompute</code>	<code>parameters.cpp</code>
<code>fields.bare_anis_prefactors</code>	<code>bare_anis_prefactors</code>	<code>parameters.cpp</code>
<code>energy.effective_anisotropy</code>	<code>effective_anisotropy</code>	<code>parameters.cpp</code>
<code>energy.critical_dmi</code>	<code>critical_dmi</code>	<code>parameters.cpp</code>
<code>energy.pma_anisotropy_field</code>	<code>pma_anisotropy_field</code>	<code>parameters.cpp</code>

(continued)

Python	C++	File
<i>Pulses</i>		
pulses.ConstantPulse	ConstantPulse	pulses.cpp
pulses.SquarePulse	SquarePulse	pulses.cpp
pulses.GaussianPulse	GaussianPulse	pulses.cpp
pulses.SuperpositionPulse	SuperpositionPulse	pulses.cpp
<i>Lattice / initial conditions</i>		
lattice.lattice_positions	lattice_positions	lattice.cpp
lattice.neighbors	inlined pbc_index stencils	lattice.hpp
initial_conditions.skyrmion_profile	skyrmion_profile	initial_conditions.cpp
initial_conditions.uniform_state	uniform_state	initial_conditions.cpp
initial_conditions.saf_skyrmion	saf_skyrmion	initial_conditions.cpp
<i>Effective field</i>		
fields.exchange_field	fused into effective_field	fields.cpp
fields.dmi_field	fused into effective_field	fields.cpp
fields.anisotropy_field	fused into effective_field	fields.cpp
fields.zeeman_field	fused into effective_field	fields.cpp
fields.rkky_field	fused into effective_field	fields.cpp
fields.effective_field	effective_field	fields.cpp
fields.effective_field_demag_pair	effective_field_demag	fields.cpp
<i>Demagnetisation</i>		
demag.precompute_demag_kernels	precompute_demag_kernels	demag.cpp
demag._precompute_slab	precompute_demag_slab	demag_slab.cpp
demag.demag_field	DemagState::compute	demag.cpp
demag_newell.precompute_demag_kernels_newell	precompute_demag_newell	demag_newell.cpp
demag_newell._aharoni_demag_factor	aharoni_demag_factor	demag_newell.cpp
demag_newell._compute_one_pair_tensor	compute_one_pair_tensor	demag_newell.cpp
demag_newell._build_layer_pair_kernel	build_layer_pair_kernel	demag_newell.cpp
<i>Energy</i>		
energy.total_energy	total_energy	energy.cpp
<i>Integrator</i>		
integrator.normalize	normalize_inplace	integrator.cpp
integrator.llgs_rhs	llgs_rhs	integrator.cpp
integrator.rhs_local_keff	RHSLocalKeff	integrator.cpp
integrator.rhs_demag	RHSDemag	integrator.cpp
integrator.rk4_step	rk4_step	integrator.cpp
integrator.rk4_step_single	rk4_step_single	integrator.cpp

(continued)

Python	C++	File
<i>Observables</i>		
main.topological_charge	topological_charge	observables. cpp
analysis.skyrmion_center	skyrmion_center	observables. cpp
analysis.skyrmion_diameter	skyrmion_diameter	observables. cpp
analysis.skyrmion_ellipse	skyrmion_ellipse	observables. cpp
analysis.dw_angle	dw_angle	observables. cpp
stochastic_llgs.diagnostics. skyrmion_center_pbc	skyrmion_center_pbc	observables. cpp
<i>Driver / I-O</i>		
main.run	run	simulation. cpp
io_ovito.write_dump	SnapshotBuffer (npz, not dump)	io_npz.cpp
<i>Sweep layer</i>		
phase_diagram.relaxation.relax	sweep::relax	sweep/relax. cpp
orchestrator.driver.run_one	sweep::run_trace	sweep/driver. cpp
orchestrator.observers.observe_ state	sweep::observe_state	sweep/ observations. cpp
orchestrator.io.save_trace	sweep::save_trace	sweep/trace_ io.cpp
orchestrator.io.load_trace	Python (bytes-metadata patch)	studies/saf_ racetrack/ orchestrator/ io.py
orchestrator.integrators.step_ deterministic	sweep::RK4LocalKeffStepper	sweep/ stepper.hpp
orchestrator.integrators.step_ demag_deterministic	sweep::RK4DemagStepper	sweep/ stepper.hpp
orchestrator.integrators.step_ stochastic	HeunStochasticStepper	stochastic/ heun.cpp
<i>Stochastic LLG</i>		
parameters_thermal.attach_thermal	attach_thermal	stochastic/ thermal.cpp
joule_heating.T_of_j	T_of_j	stochastic/ thermal.cpp
thermal_field.sample_thermal_field	ThermalRng::sample_thermal_ field	stochastic/ rng.hpp
integrator_sllg.heun_stochastic_ step	HeunStochasticStepper	stochastic/ heun.cpp
diagnostics.largest_core_mask_pbc	largest_core_mask_pbc	stochastic/ lcc.cpp
diagnostics.skyrmion_diameter_lcc	skyrmion_diameter_lcc	stochastic/ lcc.cpp
diagnostics.skyrmion_center_lcc_pbc	skyrmion_center_lcc_pbc	stochastic/ lcc.cpp
diagnostics.unwrap_trajectory	unwrap_trajectory	stochastic/ diagnostics. cpp

(continued)

Python	C++	File
diagnostics.detect_annihilation	detect_annihilation	stochastic/ diagnostics. cpp
diagnostics.hall_angle	hall_angle	stochastic/ diagnostics. cpp
experiments.run_single.trajectory_ worker	run_trajectory	stochastic/ trajectory. cpp
stochastic_llgs.io.save_trajectory	save_trajectory	stochastic/ trajectory_ io.cpp
experiments.pair_potential. skyrmion_at_position	skyrmion_at_position	stochastic/ pair_ic.cpp
experiments.pair_potential.two_ skyrmion_pair_ic	two_skyrmion_pair_ic	stochastic/ pair_ic.cpp
<i>Validation-gate helpers</i>		
validation.macros핀.make_ macros핀_params	make_macros핀_params	validation/ macros핀.cpp
validation.macros핀.run_macros핀_ ensemble	run_macros핀_ensemble	validation/ macros핀.cpp
validation.test_langevin.langevin_ function	langevin_function	validation/ analytic.cpp
validation.test_brown_reversal. brown_tau	brown_tau	validation/ analytic.cpp
validation.test_equipartition. magnon_stiffness_grid	magnon_stiffness_grid	validation/ analytic.cpp
<i>Not ported (kept in Python)</i>		
topological_torque.* (analytic Thiele)	—	—
studies/saf_race-track/scripts/ animation/*.py	— (plotting and animation stay in Python)	—

Free-boundary mask and single-layer mode. The field builders `effective_field` and `effective_field_demag`, the relaxation `sweep::relax`, and the stochastic `HeunStochasticStepper` all take an optional free-boundary mask (a row-major `ny*nx` array of 0/1 flags, or `nullptr` for the periodic path, which is byte-identical to the unmasked code). Inside a mask, out-of-region neighbours are replaced by the Rohart–Thiaville/Bogdanov–Roesler ghost-cell extrapolation shared by the exchange and DMI stencils, the field is zeroed outside the region, and the demag magnetisation is zeroed before the convolution. Single-layer mode (a lone ferromagnet) is selected by an empty bottom layer; it uses `rk4_step_single` and requires no demag kernel. These mirror the Python `mask= / m_bot=None` additions in `fields.py`, `relaxation.py`, and `integrator_sllg.py`.

Study-specific sweep analyses each build one C++ binary from a source in `src/sweeps/` (one executable per file); the public repository ships no sweep sources, which are kept locally with the study.

The stochastic experiment scans and the physics-validation gates each map to a standalone C++ binary (Tables 2 and 3).

Table 2: Stochastic experiment scan to C++ binary mapping.

Python script	C++ binary
experiments.run_single	run_single
experiments.scan_tj	scan_tj

Table 2: Stochastic experiment scan to C++ binary mapping.

Python script	C++ binary
<code>experiments.scan_arrhenius</code>	<code>scan_arrhenius</code>
<code>experiments.scan_radius</code>	<code>scan_radius</code>
<code>experiments.pair_potential</code>	<code>pair_potential</code>

Table 3: Validation gate to C++ binary mapping. Each gate prints PASS/FAIL, writes an npz under the validation output directory, and returns non-zero on failure.

Python gate	C++ binary
<code>validation.test_equipartition</code>	<code>run_equipartition</code>
<code>validation.test_t0_limit_nodemag</code>	<code>run_t0_nodemag</code>
<code>validation.test_t0_limit_demag</code>	<code>run_t0_demag</code>
<code>validation.test_langevin</code>	<code>run_langevin</code>
<code>validation.test_brown_reversal</code>	<code>run_brown_reversal</code>

The deterministic gates (`run_t0_nodemag`, `run_t0_demag`) verify that the zero-noise Heun stepper reproduces the deterministic RK4 (relative deviation $< 2\%$ on diameter, drift velocity, and topological charge; in practice the C++ runs agree to machine precision). The statistical gates (`run_equipartition`, `run_langevin`, `run_brown_reversal`) validate the thermal physics against implementation-independent closed-form targets—Rayleigh–Jeans equipartition, the Langevin function, and Brown’s high-barrier reversal time. These cannot bit-match Python because the C++ noise generator (`std::mt19937_64` with `std::normal_distribution`) differs from NumPy’s PCG64; they instead pass their own physics tolerances (measured: equipartition median ratio 0.985, Langevin RMS relative error 0.015, Brown $|\log_{10}| \leq 0.16$). The deterministic analytic targets ($L(x)$, τ_{Brown} , H_k) are parity-checked against Python in the `test_validation` unit test.

3 User guide: building and running

3.1 Dependencies

CMake ≥ 3.20 ; a C++17 compiler (AppleClang or GCC ≥ 9); FFTW3 (double precision); the vendored `libnpz` at `src/aux/libnpz`; OpenMP optional (auto-detected). Installing FFTW3:

```
brew install fftw          # macOS
apt install libfftw3-dev   # Debian / Ubuntu
```

3.2 Build

```
cd src/simulator_cpp
cmake -S . -B build
cmake --build build -j
```

This produces `build/skyrmion_sim`, one binary per sweep source in `src/sweeps/`, and the test executables.

3.3 Configuring a run

There is no command-line parsing. Every run-time value is a named variable at the top of `main()` in the relevant source file; edit and rebuild. For the standalone simulator this is `src/main.cpp`:

```
p.nx = 256; p.ny = 256; p.a = 2.0e-9;
p.dt = 5.0e-14; p.n_relax = 10000; p.n_steps = 5000;
p.pulse = std::make_shared<ConstantPulse>(4.0e11);
p.demag_kind = DemagKind::None; // None | Slab | Newell
p.dump_snapshots = true;
p.dump_every_relax = 200;
p.dump_every_drive = 100;
p.max_dump_frames = 500;
```

Then `cmake--buildbuild-j\&\&./build/skymion_sim`.

3.4 Demag model

`DemagKind::None` is the default local- K_{eff} path (slab demag folded into the anisotropy; matches the Python `rhs_local_keff` production run). `DemagKind::Slab` applies the analytic thin-film kernel via FFTW; `DemagKind::Newell` uses the mumax3-style finite-prism kernel (set `demag_accuracy` and `demag_tol_conv`; defaults 8.0 and 2×10^{-2}).

3.5 Running the sweeps

Each sweep binary has its grid baked in at the top of `main()`. Run the whole grid serially, one grid point via a SLURM array index, or many points in parallel with `xargs`:

```
./build/<sweep> # whole grid
SLURM_ARRAY_TASK_ID=3 ./build/<sweep> # one point
seq 0 8 | xargs -P 4 -I {} \
  env SLURM_ARRAY_TASK_ID={} ./build/<sweep> # local parallel
```

Outputs land under `output/`, in the same layout as the Python sweeps, so the scripts in `studies/saf_racetrack/scripts/analysis/` read them unchanged (after the one-line `load_trace` patch that accepts the C++ byte-encoded metadata).

3.6 Running the stochastic scans and validation gates

The stochastic experiment scans use the same dispatch as the sweeps: the grid is baked into `main()` and `SLURM_ARRAY_TASK_ID` selects a single point. Each writes a trajectory npz under the per-scan output directory:

```
SLURM_ARRAY_TASK_ID=0 ./build/scan_tj # one (T,j,ens) point
./build/pair_potential # whole r_init x ens grid
```

The validation gates take no arguments (the configuration is at the top of `main()`); each prints PASS/FAIL, writes its `.npz`, and returns non-zero on failure so it can gate a CI step:

```
./build/run_equipartition && ./build/run_t0_nodemag && \
  ./build/run_t0_demag && ./build/run_langevin && \
  ./build/run_brown_reversal
```

3.7 Snapshots and animation

Each grid point writes a per-frame observable trace `<id>.npz`; when `dump_snapshots=true` it also writes `<id>_snapshots.npz` containing the field history in the same schema as the simulator. Replay either with:

```
python studies/saf_racetrack/scripts/animation/animate_simulation.py # edit npz_path at top of mai
```

3.8 Threading

When OpenMP is present, per-site loops are parallel:

```
OMP_NUM_THREADS=8 OMP_PROC_BIND=close OMP_PLACES=cores ./build/skyrmion_sim
```

3.9 Tests

```
python src/simulator_cpp/tests/run_tests.py
```

This regenerates the Python references, builds, runs the 24 ctest parity suites (deterministic simulator, sweep layer, stochastic Stratonovich–Heun stepper at $T = 0$, LCC connected-component diagnostics, two-skyrmion pair IC, and the validation analytic targets), then the Python error-contract and sweep-parity harnesses. All numeric parity is at $\lesssim 10^{-12}$ relative error; the $T = 0$ stochastic stepper bit-matches the deterministic RK4.

4 Profiling: C++ vs Python (64×64)

Both implementations integrate the identical workload — a SAF skyrmion pair on a 64×64 lattice, 2000 RK4 LLGS steps after a 50-step warm-up, integration only (no I/O, no observables). Two field models are timed: the local- K_{eff} path and the slab FFT-demag path. Tools: `tools/profile_run.cpp` and `tools/profile_run.py`.

Environment. Apple M3 Pro; AppleClang 16 (`-O3-march=native`, single-threaded — OpenMP not active on this build); Python 3.12.5, NumPy 2.4.4, SciPy 1.15.2.

Table 4: Time per RK4 step (64×64 , mean of two runs).

Field model	C++ ($\mu\text{s}/\text{step}$)	Python ($\mu\text{s}/\text{step}$)	Speed-up
local- K_{eff}	≈ 316	≈ 2914	$9.2\times$
slab FFT demag	≈ 1714	≈ 7374	$4.3\times$

The local- K_{eff} path — dominated by per-site stencils and cross products — gains the most ($\sim 9\times$), since the C++ loop fuses the field terms and avoids the temporary arrays that NumPy allocates per operation. The slab path is FFT-bound; FFTW (C++) and SciPy’s `pocketfft` (Python) are comparable, so the speed-up narrows to $\sim 4\times$. With OpenMP enabled the local path would scale further on the per-site loops; FFT scaling at 64×64 is limited.

Caveats. Single trial pair, single thread, small lattice; absolute timings depend on hardware and library builds. The comparison is meant to show the order-of-magnitude advantage of the C++ integrator on the stencil-bound path, not a tuned benchmark.